University of Salerno

DEPT. OF INFORMATION ENG., ELECTRICAL ENG. AND APPLIED MATHEMATICS



Project Work – Group 10 – Artificial Vision

Advanced Vision Systems for Dynamic Pedestrian Attribute Recognition and Tracking

For corrispondence:

Registration number	Name and Surname	E-mail
0622702168	Giovanni Lamb	g.lamb@studenti.unisa.it
0622702175	Egidio Zottarelli	e.zottarelli@studenti.unisa.it
0622702328	Fabrizio Saturnino	f.saturnino@studenti.unisa.it
0622702433	Palma Orlando	p.orlando8@studenti.unisa.it

ACADEMIC YEAR 2024/2025

Summary

INTRODUCTION 3
DETECTOR: YOLOv8n 4
Architecture: The Backbone 4
Architecture: The Neck 4
Architecture: The Head 5
Innovations in YOLOv8n 5
YOLOv8n: Parameters of the predict Method 6
YOLOv8n Output 7
Choice of the YOLOv8n Model 7
TRACKING: HybridSort 9
Background: SORT and HybridSORT 9
HybridSORT Implementation 10
Choice of HybridSort 12
Choice of the Re-Identification Model 13
PAR: MultiTask Network 15
Dataset 15
Integration of VILT with VQA 16
ARCHITECTURE 18
Backbone: ResNet34 18
Binary Classification Heads 18
Combined Model 19
Optimization of Bounding Boxes for ResNet Input 19
CONFIGURATION PARAMETERS 20
INTEGRATION AND FINAL SOFTWARE 22

INTRODUCTION

In the rapidly evolving field of computer vision and artificial intelligence, the development of systems capable of interpreting and analysing video feeds in real time represents a significant step forward in applications ranging from surveillance to customer behavior analysis in retail environments.

The core of our system lies in its ability to identify individuals within a video scene, track their movements over time, and recognize important pedestrian attributes. These attributes include gender and the presence of bags or hats.

Furthermore, the project delves into the methodologies employed for tracking individuals across predefined lines and analysing their behaviour. This includes monitoring the passage of the same user over multiple lines

Additionally, the project includes a feature that allows for the analysis of the number of people and their movement direction relative to horizontal lines placed at the edges of the scene. This feature proves to be particularly useful for implementing security measures, optimizing space management, and improving services in commercial settings, as well as for the study of pedestrian flow.

DETECTOR: YOLOv8n

Object detection is a fundamental aspect of computer vision, and the **YOLO (You Only Look Once)** series stands as a milestone in this field. With YOLOv8, particularly the **YOLOv8n** (Nano) variant, the focus is on achieving an optimal balance between accuracy, efficiency, and lightweight design. This variant is specifically tailored for resource-constrained environments, such as edge devices or embedded systems, while still delivering competitive performance compared to more complex models.

YOLOv8n introduces significant improvements over previous versions by departing from some traditional paradigms and integrating new technologies to enhance adaptability, flexibility, and effectiveness in detecting objects of various sizes, shapes, and contexts. The model is divided into three main components: **backbone**, **neck**, and **head**.

Architecture: The Backbone

The backbone of YOLOv8n, based on **CSPDarknet53**, builds on the strengths of the YOLOv4 framework with significant enhancements to optimize computational efficiency and feature extraction.
The backbone is designed to:

- Reduce redundancy in the gradients of deep networks.
- Enhance the extraction of relevant features for object detection.

This architecture employs a **Cross Stage Partial Network (CSP)** strategy, which splits the computations into partial blocks to lower computational costs while maintaining high performance. Compared to traditional backbones, CSPDarknet53 improves training and inference efficiency without sacrificing accuracy.

Architecture: The Neck

The neck of YOLOv8n incorporates the **Spatial Pyramid Pooling - Fast (SPPF)** module to enhance feature extraction. The SPPF is designed to:

- Aggregate features from different spatial scales using efficient pooling operations.
- Provide a global context to improve object detection, especially in complex scenes.

- Optimize computational efficiency compared to traditional spatial pyramid pooling methods.

This module plays a crucial role in summarizing multi-scale information, ensuring the model is both accurate and fast in detecting objects.

Architecture: The Head

The **head** in YOLOv8n uses a decoupled design to improve prediction accuracy.

- **Decoupled Detection Head**:
  Instead of combining classification and bounding box regression into a single structure, the head is divided into two separate branches:
  - One branch for classification, focusing on identifying object classes.
  - Another branch for bounding box regression, which concentrates on object dimensions and locations.

This separation allows each branch to specialize in its respective task, enhancing both classification precision and localization accuracy.

Innovations in YOLOv8n

YOLOv8n incorporates several advanced techniques to improve performance:

a. Anchor-Free Design

Unlike previous YOLO models, YOLOv8n adopts an **anchor-free** approach, eliminating the need for predefined anchor boxes. This approach:

- Simplifies the detection process.
- Reduces computational costs and memory requirements.
- Improves the model's adaptability to detect objects of various shapes and sizes.

b. Data Augmentation: Mosaic 2.0 and Mixup

Mosaic Augmentation, a technique also utilized in YOLOv5, plays a key role in YOLOv8's training process. By combining multiple images from the training set into a single composite image, this method helps the model better

understand object locations in varying contexts, improving its resilience to diverse backgrounds and noise.

To maximize training efficiency, YOLOv8 automatically disables Mosaic Augmentation during the final ~10 epochs. This adjustment ensures the model focuses on refining its performance on realistic data without the distortions introduced by this augmentation technique.

YOLOv8n: Parameters of the predict Method

The predict method, used for detection, includes numerous configurable parameters. Among these, some proved particularly significant for optimizing the model's performance. Below are the key parameters, along with the adjustments made to achieve better results:

- **confidence (default: 0.25):**
  This parameter defines the minimum confidence threshold for detections to be considered valid. Objects with a confidence score below this threshold are ignored.
  - **Modification:** Increased to **0.4** to reduce the number of false positives, thereby improving detection accuracy.
- **IoU (default: 0.7):**
  The **Intersection over Union (IoU)** threshold is used for Non-Maximum Suppression (NMS), which eliminates duplicate detections within the same frame.
  - **Modification:** Reduced to **0.6** for more aggressive removal of duplicates, ensuring only one detection per person is retained.
- **imgsz (default: 640):**
  This parameter specifies the size of the images used during inference. It can be a single value (e.g., **640**, for square resizing) or a tuple (height, width).
  - **Modification:** Not increased, as larger image sizes significantly increase inference times, compromising real-time processing.
- **classes (default: None):**
  Allows filtering predictions based on a specific set of class IDs. Only objects belonging to the specified classes are included in the results.

    - **Modification:** Set to **0**, corresponding to the "person" class, as the goal is to detect people exclusively.

- **device (default: None):**
  Determines the device used for inference (CPU or GPU). The device is dynamically determined using a utility function that prioritizes the **GPU**, if available, to enhance performance.
- **half (default: False):**
  Enables half-precision inference (**FP16**) on supported GPUs, reducing memory usage and speeding up processing.
    - **Modification:** Initially set to **True**, resulting in improved inference speed. However, this led to a decrease in detection quality. To ensure reliable performance, the parameter was set to **False**.

YOLOv8n Output

The predict method, used for detecting people, returns a result object containing multiple attributes. For our purposes, we focus on specific values within the boxes attribute, namely:

- **xyxy:**
  Provides the bounding box coordinates in the **xyxy** format (top-left x, top-left y, bottom-right x, bottom-right y).
- **conf:**
  Returns the confidence scores associated with each bounding box, indicating the model's certainty in the detection.
- **cls:**
  Contains the class IDs for each detected object, allowing us to identify the class of the detected items.

These attributes are sufficient for extracting essential information about the detected people while maintaining efficiency in processing the results.

Choice of the YOLOv8n Model

To select the most suitable detector for processing videos containing people, the following models were tested: YOLO8, YOLO9, YOLO10, and YOLO11.

The YOLO9 model was quickly excluded due to its poor performance in detecting people. The other three models showed similar performance in terms of accuracy, so the choice criterion was based on the processing speed of individual frames.

The YOLO10 model registered an average processing time about 150 milliseconds longer than the YOLO8 and YOLO11 models, which showed comparable performance, including speed. Initially, the choice fell on YOLO11, considering the performance reported in the official documentation. However, during testing combined with the tracker, YOLO8 demonstrated superior performance, making it the final choice.

For the implementation, the lighter version of the model, YOLOv8n, was selected because larger models had excessively high inference times, preventing real-time processing. As a result, YOLOv8n proved to be the ideal compromise between speed and accuracy, ensuring optimal performance in real-time video processing.

TRACKING: HybridSort

Detection is effective for analyzing individual frames but faces limitations when applied to sequences of images in a video. This approach identifies objects independently in each frame, without establishing connections or correlations between their features across successive frames. Such a method can lead to challenges, particularly in complex scenarios involving occlusions, where an object is partially or completely hidden. In these cases, relying solely on frame-by-frame detection increases the risk of losing track of the object during occlusion.

Tracking, by contrast, focuses on maintaining continuity by linking an object's features across frames. This provides a more robust and consistent understanding of the object's movement over time.

A noteworthy advancement in this domain is **HybridSORT** (Hybrid Simple Online and Realtime Tracking), an evolution of **SORT** (Simple Online and Realtime Tracking). This algorithm integrates traditional tracking methods, such as the Kalman filter and the Hungarian algorithm, with advanced deep learning-based approaches. By blending classical and modern techniques, HybridSORT significantly improves tracking accuracy and robustness, making it particularly effective in challenging scenarios involving occlusions, unpredictable movements, or rapid changes in an object's appearance.

Background: SORT and HybridSORT

The HybridSORT algorithm represents an evolution of SORT, designed to enhance accuracy and robustness in object tracking. By integrating classical methods with advanced deep learning techniques, HybridSORT is particularly effective in complex scenarios such as occlusions, unpredictable movements, and rapid variations in object appearance. While retaining SORT's core approach for data association and track initialization in multi-object scenarios, HybridSORT introduces significant improvements. The algorithm is built upon four main components:

a. DETECTION:

The first phase of tracking involves identifying objects in each frame. HybridSORT leverages advanced detectors such as Faster R-CNN, YOLO, and other deep learning-based models to detect objects of interest. The information obtained from detections (bounding boxes and confidence scores) is then fed as input to the tracking module. The use of deep learning models enables HybridSORT to effectively handle partial occlusions, ensuring greater reliability compared to traditional methods.

b. ESTIMATION:

As in SORT, HybridSORT uses a constant velocity model to estimate the future positions of objects. However, this phase is enriched by the integration of neural networks, which enhance the algorithm's predictive capabilities, especially in complex conditions. The Kalman filter remains a fundamental tool for state estimation in linear systems with noise but is augmented by deep learning techniques that further refine predictions. This combined approach makes the system more robust in dynamic and unpredictable environments.

c. DATA ASSOCIATION:

The association between detections and tracks is performed using the Hungarian algorithm, as in SORT, to solve the optimization problem. However, HybridSORT improves this process by introducing visual descriptors learned through deep learning models. These descriptors, combined with classical techniques like Intersection over Union (IOU), enable more precise matching, even in the presence of partial occlusions or significant changes in object appearance. The algorithm also accounts for temporal variations in object appearance, improving its ability to maintain object identities over time.

d. TRACK IDENTITY CREATION AND DELETION:

HybridSORT dynamically and intelligently manages track identities. New tracks are created for objects that cannot be associated with existing detections, while existing tracks are removed if they remain unassociated for a defined number of frames. The addition of an AI-based layer allows for better management of transitions involving objects entering or leaving the scene. This approach ensures a continuously updated and accurate representation of tracked identities, enhancing the system's continuity and reliability.

HybridSORT Implementation

These parameters are essential configurations for the HybridSORT tracking algorithm, which is responsible for correctly associating moving object detections across different images or frames. Each parameter affects a specific aspect of the tracking process, such as the association between detections and existing tracks, handling occlusions, and the stability and reliability of the tracking. By adjusting these parameters, the behavior of the algorithm can be optimized to adapt to different scenarios, balancing tracking accuracy, occlusion handling, and computational efficiency.

det_thresh (default 0.1244260055370699, set to 0.3)*:* Defines the minimum confidence threshold for considering a detection valid. Lower values allow detections with lower confidence to be included, improving recall but increasing the risk of false positives. Higher values ensure that only more reliable detections are considered, reducing false positives but increasing the risk of excluding relevant objects.

max_age (defaul 30, set to 50)*:* Defines the maximum number of frames to keep a track alive without receiving corresponding detections. Higher values allow the system to better handle temporary occlusions or short detection losses but increase the risk of accumulating obsolete tracks. Lower values make the system more responsive, removing unupdated tracks quickly, but may lead to the loss of valid tracks.

min_hits (default 1, set to 2)*:* Defines the minimum number of consecutive detections required to confirm a new track. A higher value requires more evidence to confirm a track, reducing false positives but potentially delaying the recognition of new objects. A lower value allows faster track initiation but increases the risk of false tracks.

delta_t (default 5, set to 3)*:* Regulates the maximum number of frames in which the system can interpolate the position of an object in case of consecutive detection losses. Higher values allow better handling of temporary occlusions but may introduce errors in complex movement scenes. Lower values improve precision but reduce tracking continuity during occlusions.

asso_func (default hmiou)*:* Determines the association function used to match detections to existing tracks. Options include "iou" (Intersection over Union), based on spatial overlap, "giou" (Generalized IOU), which also considers spatial distance, and "diou" (Distance IOU), which includes the distance between the centers of the boxes. The choice of function affects the robustness of the association.

iou_threshold (default 0.3, set to 0.15)*:* Defines the IOU threshold for associating a detection with an existing track. Lower values make the association more rigorous, reducing the risk of incorrect associations but may cause track fragmentation. Higher values make associations more flexible, useful in crowded scenes, but increase the risk of errors.

Inertia (default 0.369525477649008, set to 0.1): Indicates the weight assigned to an object's velocity in its tracking. A higher value increases resistance to sudden changes in movement, useful for maintaining the track during occlusions or erratic movement. Lower values allow for more responsiveness to sudden changes, improving tracking in dynamic scenes.

TCM_first_step_weight (default 0.2866529225304586)*:* Defines the weight of the first phase of association in the Track Confidence Model (TCM). A higher value gives more importance to the first association phase, increasing the stability of the initial track. A lower value makes the system more flexible in later phases of tracking but reduces initial robustness.

longterm_reid_weight (default 0.0509704360503877, set to 1.5): Defines the weight of long-term re-identification features. Higher values improve the system's ability to handle prolonged occlusions and re-identify objects but increase computational costs. Lower values reduce computational load but compromise robustness in environments with occlusions or significant appearance changes.

use_byte (default False)*:* Indicates whether to integrate HybridSORT with the BYTE tracker. The "True" option allows leveraging both algorithms' strengths, improving tracking in complex scenarios. The "False" option uses HybridSORT independently, reducing computational complexity and resource requirements.

Choice of HybridSort

To identify the most suitable tracker for our needs, we initially tested the two trackers already implemented in the YOLO model: **ByteTrack** and **BotSort**. Between the two, BotSort demonstrated better performance than ByteTrack; however, it still exhibited some limitations, particularly in the **re-identification of previously tracked individuals** following periods of occlusion.

We then explored the **boxmot** library, which offers a broader range of trackers, including:

- **ByteTrack**,
- **BotSort** (featuring an enhanced model for object re-identification compared to the YOLO-integrated version),
- **DeepOCSort**,
- **OCSort**,
- **StrongSort**,
- **HybridSort**,
- **ImprAssoc**.

Tracker Evaluation

- **ByteTrack**: Discarded because it is not appearance-based, making it unsuitable for scenarios where individuals frequently change direction and speed.
- **DeepOCSort, OCSort, ImprAssoc**: These did not perform well in crowded environments.
- **StrongSort**: While it provided acceptable tracking accuracy, it was excluded due to its high computational cost.

The trackers that achieved a good balance between **accuracy** and **execution time** were **BotSort** and **HybridSort**. Ultimately, we chose **HybridSort**, as it demonstrated slightly better performance than BotSort, making it the most suitable solution for our operational scenario.

Choice of the Re-Identification Model

The trackers available in the **boxmot** library support dedicated **re-identification models** designed to improve long-term association and maintain object identity across frames. The library offers a variety of re-identification (RE-ID) models, including:

- **resnet50**,
- **mlfn**,
- **hacnn**,
- **mobilenetv2**,
- **osnet**,
- **osnet_ain**,
- **osnet_ibn**,
- **lmbn**,
- **clip**.

Reasons for the Choice

Among these, the **osnet** model was selected as it meets the **real-time** requirement and has been specifically trained for **person re-identification**, making it particularly suitable for our context.

While **mobilenetv2** also meets real-time performance requirements, **osnet** offers a significant advantage: its optimization for human re-identification tasks ensures greater accuracy in real-world scenarios.

Experimental Confirmation

To validate this choice, comparative tests were conducted across the various RE-ID models. The results confirmed that **osnet** provides the best balance between **accuracy** and **computational cost**, reinforcing its suitability for our application.

PAR: MultiTask Network

The topic of pedestrian attribute recognition in images is a significant challenge in various application domains, including digital signage, social robotics, business intelligence, people tracking, and multi-camera person re-identification. In these applications, there is a growing interest in simultaneously detecting a wide range of information about pedestrians.

To address this complexity, we propose the use of a Multi-Task Network, a solution particularly effective in managing the inherent challenges of pedestrian attribute recognition (**PAR**).

Dataset

To train our network for pedestrian attribute recognition, we utilized a dataset consisting of cropped images of individuals, divided into training samples (93,082) and validation samples (12,162).

Each image is fully or partially annotated with numerical labels. The presence of a negative label indicates the absence of annotations for a specific attribute. The annotated features include:

- **Gender of the person in the foreground:** the available labels are male (0) and female (1).
- **Presence of bag and hat:** the labels used are absence (0) and presence (1).

As highlighted by the data distribution analysis, the dataset exhibits several challenges typical of real-world scenarios, such as numerous missing labels and an imbalance in the distribution of information.

To address these issues, we adopted an innovative technique based on the integration of **VILT** (Vision-and-Language Transformer) with **VQA** (Visual Question Answering). This approach significantly reduced the number of missing labels, improving the dataset's quality and completeness.

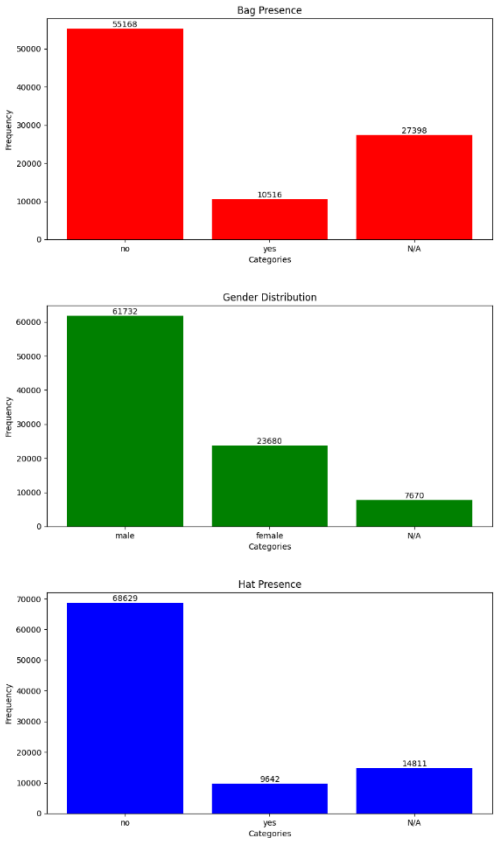


Figure 1: Dataset without annotations

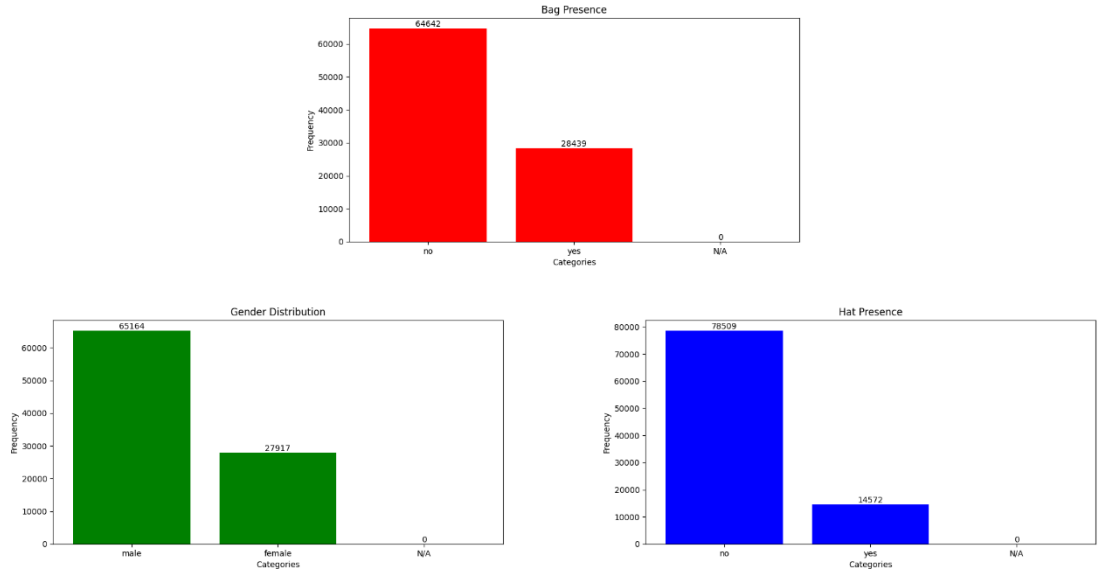


Figure 2: Dataset with annotations

Integration of VILT with VQA

The integration of **VILT** and **VQA** is built upon an advanced multimodal analysis framework, combining visual and textual processing to infer missing information.

The integration of VILT (Vision-and-Language Transformer) and VQA (Visual Question Answering) is based on an advanced multimodal analysis framework that combines visual and textual processing to infer missing information.

Feature Extraction

The system leverages VILT's ability to interpret and integrate both images and textual descriptions, generating contextually relevant questions about the visual content. These questions are then processed by VQA, which compares them with the visual data to estimate and fill in the missing annotations.

This approach relies on the system's ability to accurately interpret visual cues and respond effectively to questions, thereby "reconstructing" the missing data using both visual evidence and contextual understanding.

Conditional Update Strategy

To use VILT with VQA for completing missing annotations, a set of targeted questions was prepared to identify visible characteristics in the images. For example, if an image annotation lacks information about the person’s gender, the system automatically generates a question such as:

- "Is the person male or female?"

If the VQA component provides an answer with high confidence (e.g., above a predefined threshold for each question), this information is used to update the missing annotation.

This conditional update strategy not only enhances the accuracy of the annotations but also significantly reduces the likelihood of introducing errors into the dataset, ensuring a reliable representation of the visual characteristics present in the images.

ARCHITECTURE

This approach consists of using a single ResNet34 backbone followed by three classification output layers. The goal is to leverage weight sharing within the backbone to create an efficient feature extraction process, aiming to achieve performance comparable to a single-task model while significantly reducing inference time.

Backbone: ResNet34

The backbone is based on ResNet34, a deep convolutional network pretrained on ImageNet, used as a feature extractor. The modifications made include removing the final fully connected layer, which was originally designed for multi-class classification. As a result, the backbone acts as a feature extractor that generates a one-dimensional numerical vector (of size 512) for each input image.

This backbone leverages the deep convolutions and global pooling of ResNet34 to produce high-level representations of the images, which are subsequently utilized by the classification heads.

Binary Classification Heads

The network includes three classification heads, each responsible for a specific task. These heads share the output of the backbone as their input.

1. **GenderClassifier:**
   This head predicts the gender of the provided image. The output is a scalar value representing the probability of belonging to one of two classes (e.g., male or female).
2. **BagClassifier:**
   This head predicts the presence or absence of a bag in the image. Similarly, the output is a scalar value interpretable as a probability.
3. **HatClassifier:**
   The third head predicts the presence or absence of a hat in the image. Its functionality is analogous to the other classifiers.

Each head is designed as a single fully connected layer that takes as input the feature vector produced by the backbone and outputs a scalar value for binary classification.

Combined Model

The complete model combines the backbone with the three classification heads into a single integrated structure. The operational flow begins with the input image being processed by the backbone, which generates a feature vector. This vector serves as a numerical summary of the information extracted from the image and is subsequently passed to the three classification heads.

Each head independently computes its output. The gender classification head produces a scalar value representing the prediction related to the gender of the image. The bag classification head outputs a scalar value indicating the presence or absence of a bag. Similarly, the hat classification head generates a scalar value predicting the presence or absence of a hat in the image.

Finally, the three predictions are returned simultaneously as independent outputs, providing an efficient and modular multitasking solution.

Optimization of Bounding Boxes for ResNet Input

The bounding boxes, produced as outputs from the detection phase, are subjected to a series of transformations to meet ResNet's input requirements. These transformations include resizing and normalizing the images. Furthermore, adjustments to contrast and saturation are applied to enhance the visibility of the person’s figure and make the colors of their clothing more prominent. This preprocessing step improves the overall accuracy of the model by enhancing the quality of the input data.

CONFIGURATION PARAMETERS

To configure the camera parameters and the lines to be drawn, a configuration file named configuration.txt was used. This file contains the following details:

a. **Internal Camera Characteristics:**
These include the focal length, sensor dimensions, and image resolution.

b. **External Characteristics:**
These specify the coordinates of the camera in space, where x represents translation, y represents height, and z represents depth. Additionally, the rotation angles around the axes are defined as pitch, roll, and yaw.

c. **Line Information:**
This includes the ID and the coordinates of the start and end points of the lines.

Given these characteristics, the projectPoints function from OpenCV was chosen to project 3D points onto the image. This function requires a few specific inputs: a vector containing the coordinates of the points to be projected, a rotation vector, a translation vector, the camera matrix, and distortion coefficients.

To correctly utilize OpenCV's functions, some adjustments to the parameters are necessary. This is because OpenCV treats the z-axis as the height axis, while in the configuration file, the y-axis is used for depth.

The point coordinates are directly provided in the configuration file. As for the rotation vector, it can be obtained using OpenCV's Rodrigues function. This function takes a vector of the camera's rotation angles as input and returns a rotation matrix.

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Figure 3: Rotation Matrix

The translation vector is obtained as the product of the rotation matrix and the matrix containing the coordinates of the points to be projected.

The camera matrix, on the other hand, is defined as:

$$K = \begin{pmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{pmatrix}$$

Figure 4: Camera matrix

With fx and fy representing the width and height of the focal length in pixels, and with cx and cy being the optical center of the image, typically corresponding to the actual center. Finally, it was decided to assume no distortion. Now, all the input parameters for projectPoints are available, which will return a tuple containing the pixel coordinates for each requested point. Once the pixel coordinates are obtained, we can use the OpenCV function 'line' to draw the lines on the image.

INTEGRATION AND FINAL SOFTWARE

The input frames are passed to the YOLOv8 detector, which will return the detected bounding boxes, the detector output is then passed to the tracker in order to assign a unique id to the bounding boxes. In an iterative manner, the coordinates of each bounding box are analyzed in order, not only to track them within the image but also with the purpose of checking whether a user has passed a line and thus update what is his trajectory, in particular a point slightly above the base of the bounding box is considered in order to avoid false positives of people not completely passing the line delimited by the configuration file.

Then a copy of the frame is made and cropped at the bounding box, this new image is sent to the PAR in order to extrapolate all the features we are interested in of the detected person and correctly update its 'gender', 'bag' and 'hat' values.

Finally, the total number of people in the scene and the number of steps for each line are updated.

At the end of the video, an output file is created containing the results obtained while running the video, taking care to enter for each person the maximum value of each attribute, so that any fluctuations are ignored.

The command for starting the project is:

python main.py --video video.mp4 --configuration configuration.txt --results results.txt